# Architecture and Methodology

Problem 10: Speech Transcript Vectorization using Bag-of-Words and Collocation Features

Student: ET23BTCO127 | MT-AIML, Module 3: Word Representations and Feature Extraction | Academic year 2026-27

## 1. Objective

Voice assistants turn speech into text, but classifiers need numbers. This project builds a light, interpretable feature-extraction pipeline that converts raw speech-to-text transcripts into a Bag-of-Words (BoW) matrix **augmented with multi-word collocation features** selected by Pointwise Mutual Information (PMI). A phrase such as "credit card" then becomes its own feature instead of being split into the unrelated columns "credit" and "card".

## 2. System architecture and data flow

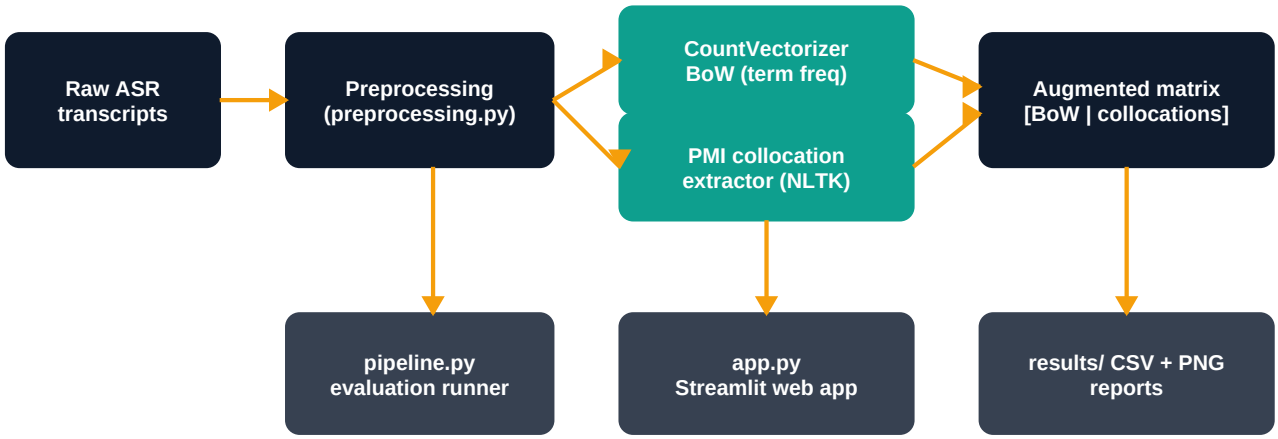



Figure 1. Components and data flow.

**Data flow.** (1) Raw transcripts (one per line, or a CSV with a text column) enter `preprocessing.py`. (2) Each transcript becomes a list of normalised tokens. (3) The token lists are used twice: `CountVectorizer` builds the BoW columns, while `CollocationExtractor` (built on `nltk.collocations`) scores adjacent word pairs with PMI. (4) Pairs that pass the minimum-frequency and PMI-threshold filters become extra columns, filled with the number of times each phrase occurs in each transcript. (5) The two blocks are stacked side by side into one sparse matrix, which feeds the Streamlit app, the evaluation pipeline and CSV exports.

## 3. Preprocessing methodology

| Step | What it does | Example |
|---|---|---|
| Lowercase | removes casing variation | PLAY -> play |
| Noise-tag removal | regex removes [noise], (cough), <unk>, *laugh* | call [noise] mom -> call mom |
| Contraction expansion | dictionary of informal forms | gonna -> going to; what's -> what is |
| Filler removal | regex for um, uh, erm, hmm, you know, i mean | turn um up -> turn up |
| Number normalisation | digits replaced by token "num" | set to 22 degrees -> set to num degrees |

| Step | What it does | Example |
|---|---|---|
| Cleaning | strips punctuation, collapses elongated letters | sooo -> soo |
| Tokenization | regex [a-z]+ | |
| Stop-word removal | NLTK English list + voice-assistant domain list (hey, please, okay ...). **Protected keywords** on, off, up, down, out, no, not, stop, play, call, set, turn ... are never removed | turn off the heater -> turn off heater |
| Lemmatization | WordNet lemmatizer (verb pass, then noun pass); small exception list | songs -> song; playing -> play |

Protecting command keywords is a design decision: NLTK treats "on", "off", "up" and "down" as stop words, which would destroy the most important phrases ("turn off", "turn up") in a voice-command domain.

## 4. Bag-of-Words representation

Let the corpus have D transcripts and vocabulary V after pruning. The BoW matrix is $X_{bow}$ in $N^{D \times V}$, where entry (d, t) is the term frequency

$$\mathbf{tf(t,\ d) = number\ of\ times\ token\ t\ occurs\ in\ transcript\ d}$$

BoW ignores word order, so it cannot tell that "credit" and "card" occurred next to each other. This is the gap collocation features fill.

## 5. Collocations and Pointwise Mutual Information

A collocation is a word sequence that occurs together more often than chance. PMI measures this by comparing the observed joint probability with the probability expected if the two words were independent:

$$\mathbf{PMI(x,\ y) = \log2\ [\ P(x,\ y)\ /\ (\ P(x)\ *\ P(y)\ )\ ]}$$

Probabilities are estimated by relative frequency, as in `nltk.collocations`: P(x,y) = c(x,y)/N and P(x) = c(x)/N, where c(.) are counts and N is the total number of tokens. For three words the same idea is used: PMI(x,y,z) = log2 [ P(x,y,z) / (P(x) P(y) P(z)) ] (NLTK's trigram PMI).

**Worked example from the sample dataset** (N = 1127 tokens): "credit card" occurs c(credit, card) = 9 times; c(credit) = 10; c(card) = 9. Therefore PMI = log2( 9 x 1127 / (10 x 9) ) = log2(112.7) = **6.82 bits**, which is the value the software reports. The words almost always appear together.

**Interpretation.** PMI = 0: independent. PMI > 0: attract each other. PMI < 0: avoid each other. Each +1 means the pair is twice as likely as chance.

**Known weakness.** If both words are rare, even a single co-occurrence gives a very large PMI. Rare pairs are therefore ranked above genuinely useful frequent phrases. The standard remedy, used here, is a **minimum frequency filter** before scoring.

## 6. Feature matrix design

The final matrix is the horizontal concatenation of two blocks:

$$\mathbf{X = [\ X_{bow}\ |\ X_{col}\ ]\ in\ N^{D\ x\ (V\ +\ K)}}$$

K is the number of collocations that survive the filters. Column j of $X_{col}$ holds, for each transcript, the number of times collocation j occurs as an adjacent pair (or triple) in that transcript. Collocation columns are named "word1_word2" (for example `credit_card`), BoW columns keep the plain word. Both blocks are counts, so a transcript containing "credit card" has +1 in *credit*, +1 in *card* and +1 in *credit_card*. The matrix is stored as a SciPy sparse CSR matrix.

**No data leakage.** The vocabulary and the PMI scores are learned in `fit()` from training transcripts only. `transform()` then maps unseen transcripts onto those fixed columns; unknown words and unseen phrases are ignored. The evaluation runner fits the vectorizer inside every cross-validation fold.

## 7. PMI thresholding

**Why a threshold?** A transcript corpus contains hundreds of adjacent pairs, most of them accidental. Keeping every pair multiplies the number of columns and adds noise. Keeping only pairs with PMI >= t retains the strongly associated ones.

| PMI threshold t | Effect |
|---|---|
| t <= 0 | almost every frequent pair is kept; widest matrix, many accidental pairs |
| t = 2 to 4 (default 3) | balanced; drops pairs that only co-occur because both words are very common |
| t >= 6 | only very exclusive pairs; narrow matrix but frequent useful phrases may be lost |

**Observed behaviour** (350 transcripts, min_freq = 5):

| PMI threshold | -1 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|---|
| Collocation columns | 65 | 65 | 65 | 64 | 63 | 60 | 46 | 36 | 17 |
| Total columns | 229 | 229 | 229 | 228 | 227 | 224 | 210 | 200 | 181 |

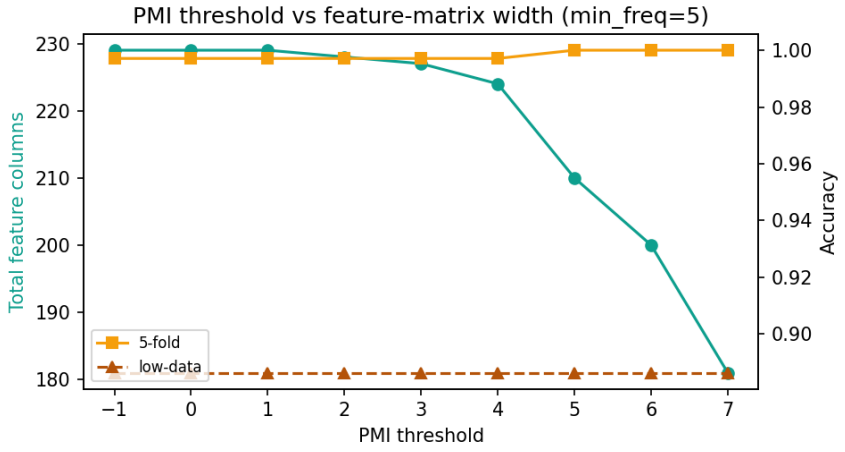



Figure 2. Matrix width and accuracy versus PMI threshold.

A subtle point: "turn up" occurs 18 times but has PMI = 2.98, just under the default threshold, because "turn" and "up" are both very common words. A frequent pair can have low PMI, so PMI alone does not measure usefulness. It measures *exclusivity*.

## 8. Vector dimensionality management

| Control | Applies to | Effect |
|---|---|---|
| min_df | BoW | drops words that appear in fewer than min_df transcripts (removes typos / ASR errors) |
| max_features | BoW | keeps only the most frequent words |
| min_freq | Collocations | ignores pairs seen fewer than min_freq times (fixes PMI rare-pair bias) |
| pmi_threshold | Collocations | keeps only strongly associated pairs |
| max_collocations | Collocations | hard cap, keeps the highest-PMI pairs |
| binary | Both | presence/absence instead of counts |
| Sparse storage | Matrix | only non-zero entries stored (sparsity about 98% on the sample data) |

| min_freq | 1 | 2 | 3 | 5 | 8 |
|---|---|---|---|---|---|
| Collocation columns (PMI >= 3) | 214 | 147 | 100 | 63 | 20 |

Table: increasing min_freq from 1 to 8 shrinks the collocation block from 214 to 20 columns.

## 9. Computational complexity

Let T be the total number of tokens and U the number of distinct n-grams. Preprocessing is O(total characters). Counting n-grams is O(T) per n-gram size. PMI scoring is O(U). Building the sparse matrix is O(T) because each transcript is scanned once per n-gram size and each hit is one dictionary lookup. Memory is O(non-zero entries), not O(D x (V + K)).

## 10. Limitations

- Collocations are found only between **adjacent** tokens after stop-word removal; this can create artificial pairs (see Evaluation Report, failure analysis).
- PMI is biased towards rare events even with the frequency filter; results depend on min_freq.
- English only; transcripts must already exist (no speech recognition is performed).
- The bundled dataset is synthetic (template-generated) and is meant to demonstrate the pipeline, not to claim real-world accuracy.